

UNIT V

5.0 INTRODUCTION

The basic data types in C are int, float and char. using these data types, we can derive some other data types, the data types that are derived from the basic data types are called derived data types.

We have already discussed three of the six derived types: arrays, functions and pointers. In this unit, we discuss the three remaining derived types: structure, union and enumerated. We will also discuss the use of type definition and Bit field. The derived data types are shown in Figure 5.0.

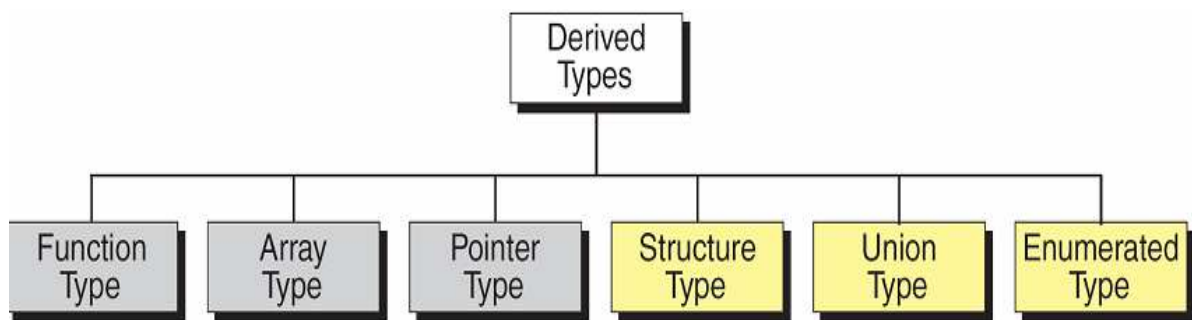


Figure 5.0 Derived Types in C

5.1 STRUCTURES

We have seen that arrays can be used to represent a group of data items that belong to the same type. However in real world, we often deal with entities that are collection of dissimilar data items. Using array variables we cannot store different items under one name. C supports a derived data type known as structure, which is a method for storing data of different types.

Structure Definition

A structure is a collection of logically related elements, possibly of different types, having a single name.

Advantages

- ❖ Structures help to organize complex data in a more meaningful way.
- ❖ Being able to manipulate several variables as a single group makes your programs easier to manage.
- ❖ Business data processing uses the concepts of structures in almost every program. Example, student database and employee database management.

Good candidates for structures:

Time: year, month, day, hour, minutes, second

Complex number: real part, imaginary part

Point in a 2-dimensional plane: x axis, y axis

5.1.1 DEFINITION OF STRUCTURES

As variables are defined before they use in the program, structures are also defined and declared before they are used.

A structure definition forms a template that may be used to create structure objects. The variables that make up the structure are called members of the structure.

A structure can be defined using three different ways,

- ❖ Tagged Structure
- ❖ Structure Variables
- ❖ Typedef Structure

Tagged Structure

The structure definition associated with the structure name is referred as tagged structure. It does not create an instance of a structure and Does not allocate any memory.

The general form or syntax of tagged structure definition is as follows,

```

struct tag_name
{
    type1 member1;
    type2 member2;
    .....
    .....
};

```

Where,

- ❖ **struct** is the keyword which tells the compiler that a structure is being defined.
- ❖ **Tag_name** is the name of the structure.
- ❖ member1, member2 ... are called members of the structure. The members are declared within curly braces.
- ❖ The closing brace must end with the semicolon.

Example: student details using tagged structure

```

struct student
{
    char name [10];
    int roll_number;
    float avg_marks;
};          // no memory is allocated for the structure

```

Note the following points with respect to above structure definition,

- ❖ **struct** is the keyword which tells the compiler structure is being defined.
- ❖ **student** is an identifier representing the structure name (tag_name).
- ❖ name, roll_number and avg_marks are members of a structure and are themselves are not variables. They do not occupy any memory.

Memory is not reserved for the structure definition since no variables are associated with the structure definition. The members of the structure do not occupy any memory until they are associated with the structure variables.

The declaration of the structure variable takes of the form,

```
struct tag_name var1, var2...;
```

where, struct is the keyword. Tag_name is the name of the structure. Structure variables are separated by comma, followed by semicolon.

We can declare structure variables any where in the program.

For the above example the variable declaration as follows,

```
struct student s1; // memory is allocated for the variable
```

now a variable of type **struct student** (derived type) is created, the memory is allocated for the variable s1.

The following figure shows the memory organization for the above example.

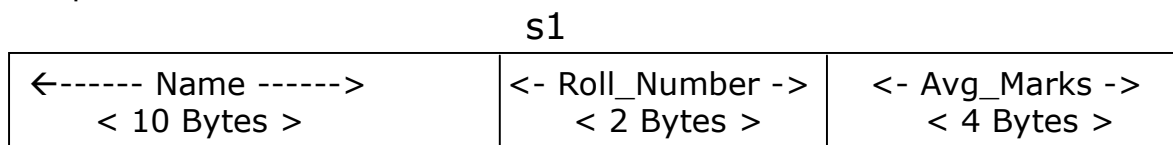


Figure 5.1 Memory map for structure variable

The number of bytes allocated for the structure variable is the sum of sizes of the individual members. In the above example the size of s1=16 bytes (10+2+4).

Note: Normally, structure definition appears at the beginning of the program file, before any variables or functions defined.

Structure Variables

Here we are combining both the template declaration and variable declaration in one statement, is referred as structure variables. The syntax of tagged structure is as follows,

```
struct tag_name
{
    type1 member1;
    type2 member2;
    .....
} var1, var2...;
```

Where,

- ❖ **struct** is the keyword which tells the compiler that a structure is being defined.
- ❖ **tag_name** is the name of the structure.
- ❖ member1, member2 ... are called members of the structure. The members are declared within curly braces.
- ❖ **var1, var2...** are structure variables that follow curly braces. Here each variable occupies memory.
- ❖ The closing brace must end with the semicolon.

Example: student details using structure variables

```
struct student
{
    char name [10];
    int roll_number;
    float avg_marks;
} s1;
```

Here, name, roll_number and avg_marks are the structure members. s1 is the structure variable, the compiler allocates memory for the variable s1 as shown in Figure 5.1.

We can omit the tag name from the structure definition, the following is valid.

```
struct
{
    char name [10];
    int roll_number;
    float avg_marks;
} s1, s2;
```

Declares s1 and s2 as structure variables representing two students. This structure definition and declaration is normally not used because we can not declare variables in later stage in the program.

Typedefined Structure

- ❖ The structure definition associated with keyword typedef is called type-defined structure.
- ❖ This is the most powerful way of defining the structure.

- ❖ The syntax of typedefed structure is

```
typedef struct
{
    type1 member1;
    type2 member2;
    .....
} TYPE_ID;
```

where,

- ❖ typedef is keyword added to the beginning of the definition.
- ❖ struct is the keyword which tells the compiler that a structure is being defined.
- ❖ member1, member2...are called fields of the structure.
- ❖ The closing brace must end with type definition name which in turn ends with semicolon.

Example: Student details

```
//Structure Definition
typedef struct
{
    char name [10];
    int roll_number;
    float avg_marks;
} STUDENT; //no memory is allocated for structure

/* structure declaration */
STUDENT s1, s2; //memory is allocated for the variables.
```

Note: Using typedef it is not possible to declare a variable. But, we can have user defined data type. TYPE_ID can be treated as the new data type.

5.1.2 INITIALIZATION OF STRUCTURES

The rules for structure initialization are similar to the rules for array initialization. The initializers are enclosed in braces and separate by commas. They must match their corresponding types in the structure definition.

The syntax is shown below,

struct tag_name variable = {value1, value2,... valuen};

Structure initialization can be done in any of the following initialization.

Initialization along with Structure definition

Consider the structure definition for student with three fields name, roll number and average marks. The initialization of variable can be done as shown below,

```
struct student
{
    char name [5];
    int roll_number;
    float avg;
} s1= {"Ravi", 10, 67.8};
```

The various members of the structure have the following values.

←--- name -----> ← roll_number -> ←----- avg ----->

R	a	v	i	\0	1	0	6	7	.	8
----------	----------	----------	----------	-----------	----------	----------	----------	----------	----------	----------

Figure 5.2 Initial Value of S1

Initialization during Structure declaration

Consider the structure definition for student with three fields name, roll number and average marks. The initialization of variable can be done as shown below,

```
struct student
{
    char name[5];
    int roll_number;
    float avg;
};

struct student s1= {"Ravi", 10, 67.8};
```

The various members of the structure have the values as shown in Figure 5.2.

We can also initialize multiple variables using comma between variables.

Points to Remember

- ❖ The members of the structure cannot be initialized in the structure definition.

For example,

```
struct s
{
    char name [10] ="ravi";
    int rno;
} s1;
```

is invalid.

- ❖ The initial values are assigned to members of the structure on one-to-one basis i.e., the values are assigned to various members in the order specified from the first member.

For example,

```
s1= {"ravi", 60, 56.7};
```

is valid. Here, the string "ravi" will be assigned to the first member. 60 is assigned to the second member and 56.7 is assigned to the third member.

- ❖ During partial initialization, the values are assigned to members in the order specified and the remaining members are initialized with garbage values.

For example,

```
s1= {"ravi"};
```

will assign the string "ravi" to name and the remaining members are initialized to garbage values.

However the statement,

```
s1= {40, 67.8};
```


is invalid, because, without initializing the first member name, we are trying to initialize last two members.

- ❖ During initialization, the number of initializers should not exceed the number of members. It leads to syntax error.

For example,

```
s1= {"ravi", 45, 78.9, 89};
```

the compiler issues a syntax error "too many initializers"

5.1.3 ACCESSING STRUCTURES

We know that variables can be accessed and manipulates using expressions and operators. On the similar lines, the structure members can be accessed and manipulated. The members of a structure can be accessed by using dot(.) operator.

dot (.) operator

Structures use a dot (.) operator to refer its elements, also known as period operator.

Before dot, there must always be a structure variable. After the dot, there must always be a structure element.

The syntax to access the structure members as follows,

```
structure_variable_name. structure_member_name
```

For example, consider the example as shown below,

```
struct student
{
    char name [5];
    int roll_number;
    float avg;
};

struct student s1= {"Ravi", 10, 67.8};
```

The members can be accessed using the variables as shown below,

s1.name --> refers the string "ravi"

s1.roll_number --> refers the roll_number 10
s1.avg --> refers avg 67.8

Arrays with in structures

It is also possible to declare an array as a member of structure, like declaring ordinary variables.

For example to store marks of a student in three subjects the we can have the following definition of a structure.

```
struct student
{
    char name [5];
    int roll_number;
    int marks [3];
    float avg;
};
```

Then the initialization of the array marks done as follows,

```
struct student s1= {"ravi", 34, {60,70,80}};
```

The values of the member marks array are referred as follows,

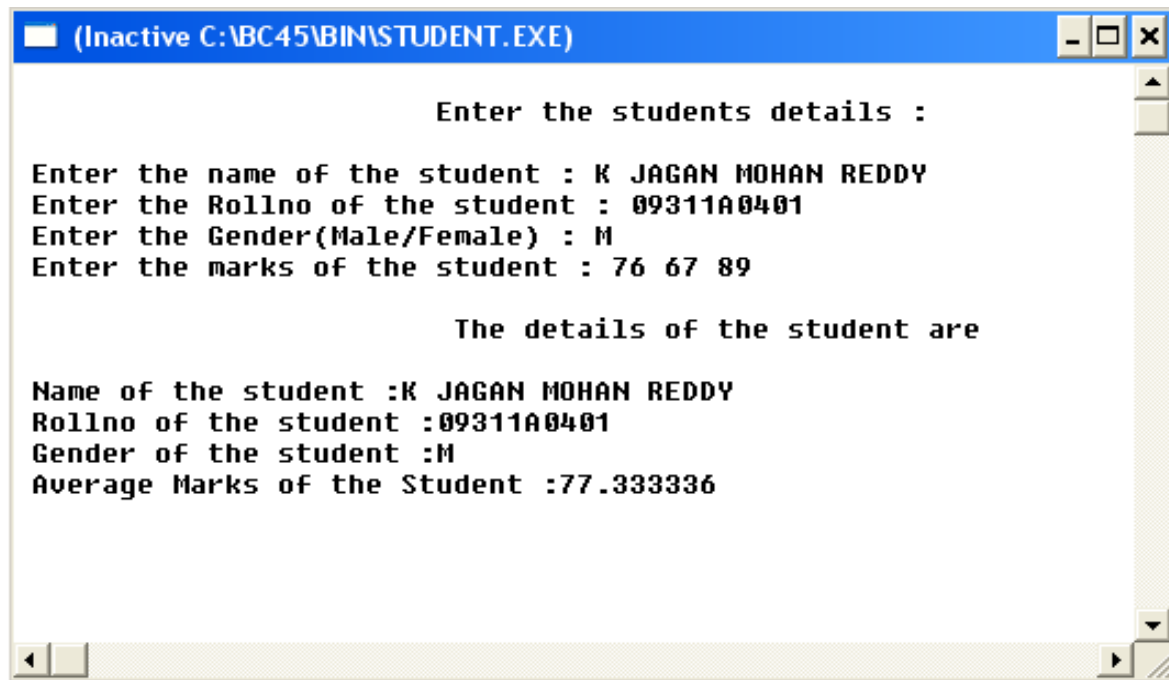
s1.marks [0] --> will refer the 0th element in the marks
s1.marks [1] --> will refer the 1st element in the marks
s1.marks [2] --> will refer the 2ndt element in the marks

The below program illustrates how to define a structure, declare a structure and access the members of the structure.

```

#include<stdio.h>
struct student
{
    char name[10];
    char rollno[10];
    char gender;
    int marks[3];
    float avg;
};
int main()
{
    int i,sum=0;
    struct student s1;
    printf("\n\t\t\tEnter the students details : ");
    printf("\n\n Enter the name of the student : ");
    fflush(stdin);
    gets(s1.name);
    printf(" Enter the Rollno of the student : ");
    gets(s1.rollno);
    printf(" Enter the Gender(Male/Female) : ");
    scanf("%c",&s1.gender);
    printf(" Enter the marks of the student : ");
    sum=0;
    for(i=0;i<3;i++)
    {
        scanf("%d",&s1.marks[i]);
        sum=sum+s1.marks[i];
    }
    s1.avg=sum/6.0;
    printf("\n\t\t\tThe details of the student are ");
    printf("\n\n Name of the student :%s",s1.name);
    printf("\n Rollno of the student :%s",s1.rollno);
    printf("\n Gender of the student :%c",s1.gender);
    printf("\n Average Marks of the Student :%f",s1.avg);
    return 0;
}

```



5.2 STRUCTURE OPERATIONS

Any operation that can be performed on ordinary variables can also be performed on structure members. But, some operations can not be performed on structure variables. The following sections deals with operations that carried on structure and structure members.

Copying of structure Variables or members

Copying of two structure variables is achieved using assignment operator. But, one structure variable can be assigned to another structure variable of the same type.

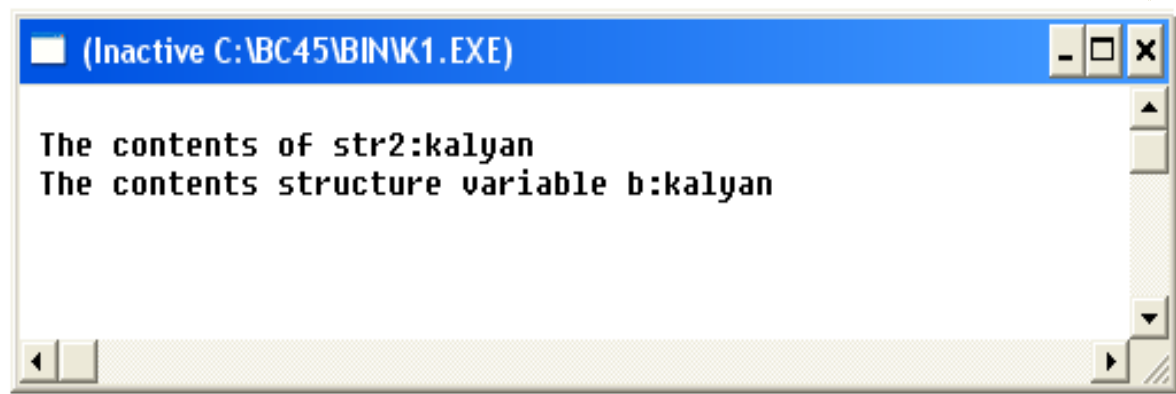
Example,

```
struct student
{
    char name [10];
    float avg;
} a, b;
```

```
a=b; //copies b in to a
a.avg = b.avg; // copies b average in to a.avg
```

We can not copy the variables of different structure type.

```
#include<stdio.h>
#include<string.h>
struct s
{
    char name[10];
};
int main()
{
    char str1[10]="kalyan",str2[10];
    struct s a={"kalyan"},b;
    strcpy(str2,str1); // string copying
    b=a;               // structure copying
    printf("\n The contents of str2:%s",str2);
    printf("\n The contents structure variable b:%s",b.name);
    return 0;
}
```



Comparison of two structure variables or members

Two variables of same structure type or dissimilar type is not allowed. For example, the following operations are invalid, even though s1 and s2 are of the same type,

s1==s2; (or) s1! = s2; invalid: because comparison is not allowed between structure variables.

However, the members of two structure variables of same type can be compared the same way as ordinary variables. s1 and s2 are two structures, and then the following operations are valid.

<i>Operation</i>	<i>Meaning</i>
s1.member1 == s2.member1	Compares member1 of a with member1 of b and return true if they are same, false otherwise.
s1.member1 != s2.member1	returns if the member1 of a and member1 of b are not same and false otherwise.

Table 5.1 Comparison of Structure Members

Arithmetic Operations on Structures

The members of a structure are same to any ordinary variable and so any operation that can be performed on a variable can also be performed on structure members. The following operations are valid.

<i>Expression</i>	<i>Meaning</i>
++e.salary	increments the salary before accessing its value.
e.salary++	increments the salary after accessing its value.
&e.salary	Access the address of e.salary.
&e	Access the beginning address of the structure.

Table 5.2 Arithmetic Operations on Structure Members

Note: The arithmetic, relational, logical and other various operations can be performed on individual members of the structures but not on structure variables.

5.3 NESTED STRUCTURES

A structure which includes another structure is called nested structure i.e a structure can be used as a member of another structure.

The syntax for the nesting of the structure as follows,

```
struct tag_name1
{
    type1 member1;
    .....
    .....
};

struct tag_name2
{
    type1 member1;
    .....
    .....
    struct tag_name1 var;
    .....
};
```



The syntax for accessing members of a nested structure as follows,

outer_structure_variable.innerstructurevariable.membername

Good candidates for nested structures:

Student: dob -> day, month, year
 Employee: allowance -> da, ta, hra
 Player: tests, one days, T20

Example: Consider the student information name, roll no, DOB and avg. The DOB consists of day, month, and year. It can be defined as shown below,

```

struct data
{
    int day;
    int month;
    int year;
};

struct student
{
    char name [10];
    int roll_number;
    struct data dob;
    int marks [3];
    float avg;
} s1;

```

Note that the above structure consists of a member identified by `dob` whose type is `struct data`.

The members contained in the inner structure namely `day`, `month` and `year` can be referred to as

```

s1.dob.day
s1.dob.month
s1.dob.year

```

An inner-most member in a structure can be accessed by chaining all the concerned structure variables (from outer-most to inner-most).

The memory organization of variable `s1` is shown in Figure 5.3.

s1

name	roll_number	dob			marks	year
10 bytes	2 bytes	day	month	year	6 bytes	4 bytes
		2 bts	2 bts	2 bts		

Figure 5.3: Memory organization of nested structure

We can nest two structures by using the following syntax,

```
struct tag_name1
{
    type1 member1;
    .....
    .....
    struct tag_name2
    {
        type1 member1;
        .....
        .....
    } var;
};
```

Example

```
struct student
{
    char name [10];
    int roll_number;
    struct data
    {
        int day;
        int month;
        int year;
    } dob;
    int marks [3];
    float avg;
};
```

In the above definition it is mandatory to initialize variable to the inner most structure. The members are accessed same way as first method.

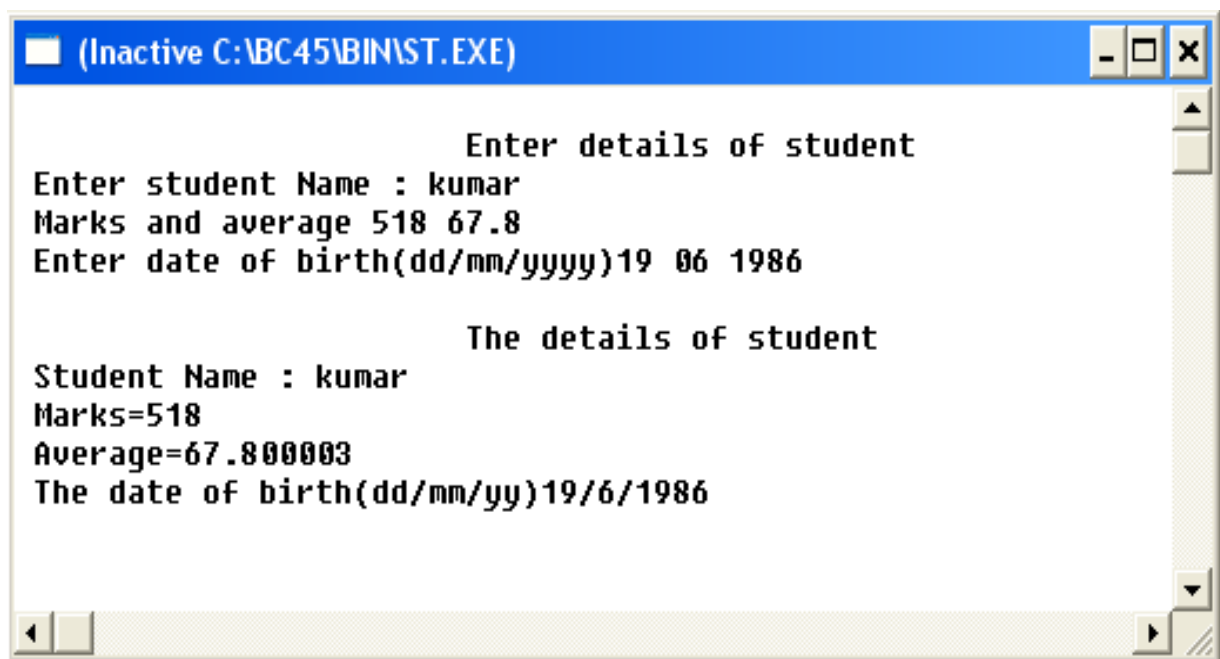
Below program illustrates nesting of structures.

```
#include<stdio.h>
struct student
{
    char name[10];
    int rno;
    float avg;
    struct data
    {
        int day;
        int month;
        int year;
    }dob;
};
int main()
{
    struct student s1;
    printf("\n          Enter details of student ");
    printf("\n Enter student Name : ");
    gets(s1.name);
    printf(" Marks and average ");
```

```

scanf("%d%f",&s1.rno,&s1.avg);
printf(" Enter date of birth(dd/mm/yyyy)");
scanf("%d%d%d",&s1.dob.day,&s1.dob.month,&s1.dob.year);
printf("\n                The details of student ");
printf("\n Student Name : ");
puts(s1.name);
printf(" Marks=%d \n Average=%f ",s1.rno,s1.avg);
printf("\n The date of birth(dd/mm/yy)");
printf("%d/%d/%d",s1.dob.day,s1.dob.month,s1.dob.year);
return 0;
}

```



5.4 ARRAY OF STRUCTURES

As we have an array of integers, we can have an array of structures also. For example, suppose we want to store the information of class of students, consisting of name, roll_number and marks, A better approach would be to use an array of structures.

Array of structures can be declared as follows,

```
struct tag_name arrayofstructure[size];
```

After having the above declaration a list of variables of type struct tag_name.

Let's take an example, to store the information of 3 students, we can have the following structure definition and declaration,

```
struct student
{
    char name[10];
    int rno;
    float avg;
};
```

```
struct student s[3];
```

Defines array called **s**, which contains three elements. Each element is defined to be of type struct student.

For the student details, array of structures can be initialized as follows,

```
struct student s[3]={{"ABC",1,56.7},{"xyz",2,65.8},{"pqr",3,82.4}};
```

The memory organization of the variables is illustrated in the below Figure,

S			
s [0]	Name (10bytes)	rno (2 bytes)	avg (4bytes)
s [1]	Name (10bytes)	rno (2 bytes)	avg (4bytes)
s [2]	Name (10bytes)	rno (2 bytes)	avg (4bytes)

Figure 5.4: Memory organization of array of structures

```
// program illustrates reading and displaying of student information for a class
of students using array of structures
#include<stdio.h>
struct student
{
    char rollno[10];
    char name[20];
    char branch[10];
    char gender;
    struct birthday
    {
        int day;
        char month[10];
        int year;
    }DOB;
    int sub[3];
    float avg;
};

int main()
{
    int i,n,j,sum=0;
    struct student s[10];
    printf(" Enter how many students are there in the class:");
    scanf("%d",&n);
    printf("\t\t\t\t Enter %d student's details",n);
    for(i=0;i<n;i++)
    {
        printf("\n Enter Roll number of %d Student:",i);
        fflush(stdin);
        gets(s[i].rollno);
        printf(" Enter the name:");
        gets(s[i].name);
        printf(" Enter Branch:");
        gets(s[i].branch);
        printf(" Enter Gender:");
        scanf("%c",&s[i].gender);
        printf(" Entre DOB:");
        scanf("%d%s%d",&s[i].DOB.day,s[i].DOB.month,&s[i].DOB.year);
        printf(" Enter the marks of student:");
    }
}
```

```

        sum=0;
        for(j=0;j<3;j++)
        {
            scanf("%d",&s[i].sub[j]);
            sum=sum+s[i].sub[j];
        }
        s[i].avg=sum/3.0;
    }
    printf("\n*****");
    printf("\n\t\t\t Student details:");
    printf("\n*****");
    for(i=0;i<n;i++)
    {
        printf ("\n Student %d:",i+1);
        printf ("\nRoll number:%s\n",s[i].rollno,s[i].name);
        printf ("\nDOB: %d-%s-%d",s[i].DOB.day,s[i].DOB.month,s[i].DOB.year);
        printf ("\nAverage=%f", s[i].avg);
    }
    return 0;
}

```

5.5 STRUCTURES AND POINTERS

The way we can have a pointer pointing to an int, or a pointer pointing to a char, similarly we can have a pointer pointing to a struct. Such pointers are known as 'structure pointers'.

To access the values of the members of the structure using pointers we need to perform following things,

1. Declare structure variable: struct tag_name var;
2. Declare a pointer to a structure: struct tag_name *ptr;
3. Store the address of structure variable in structure pointer:
ptr=&var;
4. Access the members of the structure.

Members of the structures can be accessed in following ways.

Using De-Reference Operator * and Dot (.) Operator

If ptr is a pointer to a structure, then the structure itself can be accessed using indirection operator as shown below,

struct tag_Name *ptr; // Refers to the whole structure

Once the structure pointer is initialized, then each member of the structure can be accessed using dot operator as shown below,

```
(*ptr).structure_member1
(*ptr).structure_member2
...
```

Let's take the following segment of code

```
struct student
{
    int rno;
    char name[10];
    float avg;
};

struct student s1={101,"ABC",78.98};
struct student *ptr;
ptr=&s1;
```

Now the members of the structure can be accessed by using the dot operator as shown below,

```
(*ptr).name    // Access the name
(*ptr).rno      // Access roll number
(*ptr).avg      // Access average
```

Note: The parenthesis to all three expressions is necessary. We should not omit the parenthesis. For example,

```
*ptr .name    // invalid way of accessing the member
```

Using Selection Operator (->)

If ptr is a pointer to structure, then the members of the structure can also be accessed using selection operator denoted by ->(which is formed by minus sign and greater than symbol).

Using this various members of the structure can be accessed as shown below,

ptr -> structrure_member

For the above example the members can be accessed as follows,

```
ptr - > name    // Access the name
ptr - > rno     // Access roll number
ptr - > avg     // Access average
```

Remember that on the left hand side of the dot operator, there must always be a structure variable, whereas on the left hand side of -> operator there must be always be a pointer to structure. This method is efficient, preferred over the previous method.

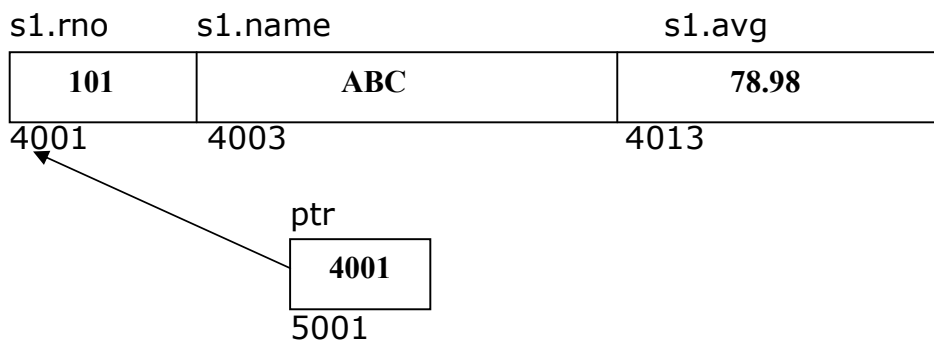
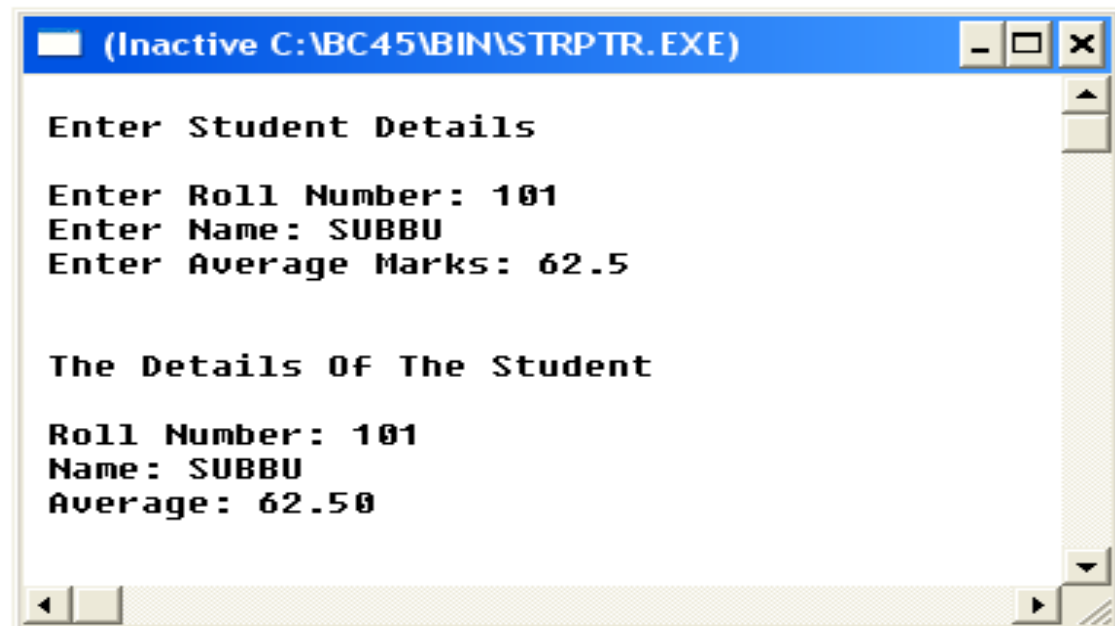


Figure 5.5: Memory Representation of Structure Pointer

The arrangement of structure variable and pointer to structure in memory is as shown in the Figure 5.5,

// C program to read and display student details using pointer to structure

```
#include<stdio.h>
struct student
{
    int rno;
    char name[10];
    float avg;
};
int main()
{
    struct student s1;
    struct student *ptr;
    ptr=&s1;
    printf("\n Enter Student Details \n\n");
    printf(" Enter Roll Number: ");
    scanf("%d",&ptr->rno);
    printf(" Enter Name: ");
    fflush(stdin);
    gets(ptr->name);
    printf(" Enter Average Marks: ");
    scanf("%f",&ptr->avg);
    printf("\n\n The Details Of The Student \n");
    printf("\n Roll Number: %d",ptr->rno);
    printf("\n Name: %s",ptr->name);
    printf("\n Average: %.2f",ptr->avg);
    return 0;
}
```

Structure Containing Pointers

We can also contain pointers as members of the structures. The pointer is pointing to a value of the structure member or some other variable. In general to reduce memory wastage we are using this concept to declare strings as structure members using character pointer.

The below code illustrates pointers as members of structures.

```
#include<stdio.h>
struct student
{
    char name [10];
    int rno;
    float *ptr;
};
int main ()
{
    struct student s1= {"ABC",405};
    float avg=34.5;
    s1.ptr=&avg;           // initializing pointer
    printf ("%f",*s1.ptr); // accessing the value of a variable
    return 0;
}
```

5.6 STRUCTURES AND FUNCTIONS

We should note that structures are more useful if we are able to pass them to functions and return them. The structures members can be passed to the function in various ways as shown below,

1. BY passing individual members of structure
2. By passing the whole structure
3. By passing structures through pointers

Passing Individual Members

The first method is to pass each member of the structure as an actual argument of the function call. The actual arguments are treated independently like ordinary variables.

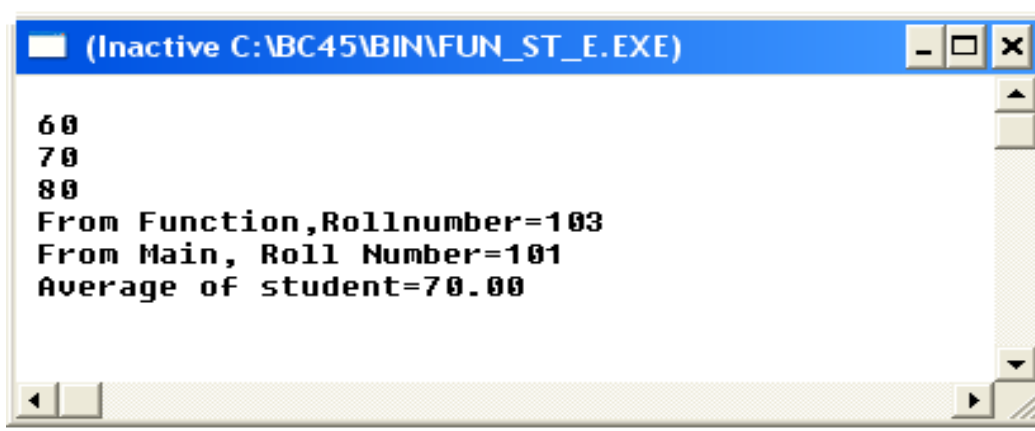
// Passing individual structure elements

```
#include<stdio.h>
struct student
{
    char name[10];
    int rno;
    int marks[10];
    float avg;
};
void display(int ,int [],float * );
int main()
{
    struct student s1={"ABC",101,{60,70,80}};
    display(s1.rno,s1.marks,&s1.avg);
    printf("\n From Main, Roll Number=%d",s1.rno);
    printf("\n Average of student=%.2f",s1.avg);
}
void display(int rno,int marks[],float *pavg)
{
    int i,sum=0;
    for(i=0;i<3;i++)
```

```

{
    printf("\n %d",marks[i]);
    sum=sum+marks[i];
}
rno=rno+2;
printf("\n From Function,Rollnumber=%d",rno);
*pavg=sum/3.0;
}

```



Explanation

Observe that in the declaration of the structure, rno is declared as int, marks are declared as integer array and avg as float. Therefore, when we call the function display using

display (s1.rno, s1.marks, &s1.avg);

We are passing the structure individual member value, the base address of the array marks and the address of structure member. Thus, this is a mixed of call-a call by reference as well as a call by value.

Note: This method is inefficient when the structure size becomes large. A better way would be to pass the entire structure variable at a time.

Passing Whole Structure

The second method involves passing a copy of the entire structure to the called function. Any changes to structure members within the function are not reflected in the original structure. It is therefore, necessary for the function to return the entire structure back to the calling function.

The general format of sending a copy of a structure to the called function is:

return_type function_name (structure_variable_name);

The called function takes the following form:

```
data_type function_name(struct_type tag_name)
{
    .....
    .....
    return(expression);
}
```

The called function must be declared for its type, appropriate to the data type it is expected to return.

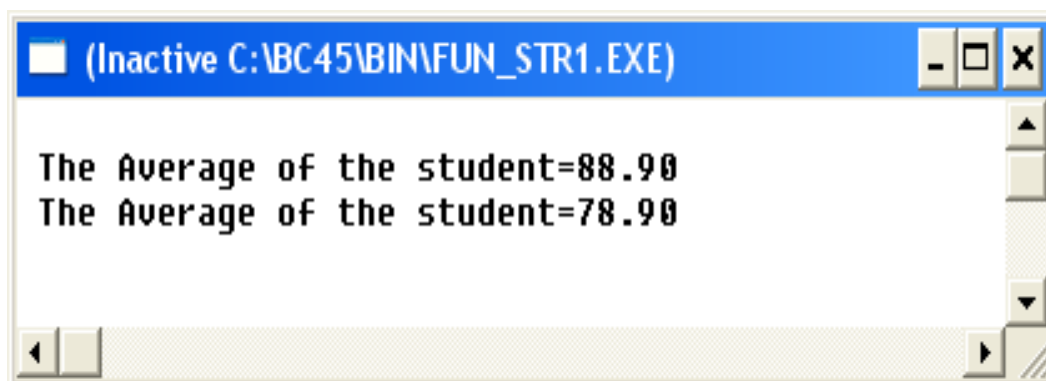
The structure variable used as the actual argument and the corresponding formal argument in the called function must of the same struct type.

The return statement is necessary only when the function is returning some data back to the calling function. The expression may be any simple variable or structure variable or an expression using simple variables.

When a function returns a structure, it must be assigned to a structure of identical type in the calling function. The called functions must be declared in the calling function appropriately.

// Passing Entire Structure

```
#include<stdio.h>
struct student
{
    char name[10];
    int rno;
    float avg;
};
void fun(struct student );
int main()
{
    struct student s1={"SASIDHAR",101,78.9};
    fun(s1);
    printf("\n The Average of the student=%.2f ",s1.avg);
    return 0;
}
void fun(struct student s)
{
    s.avg=s.avg+10;
    printf("\n The Average of the student=%.2f ",s.avg);
}
```



Explanation

Note that here the calling of function fun becomes quite compact, fun (s1); Then the formal argument in the called function defined as struct type struct student.

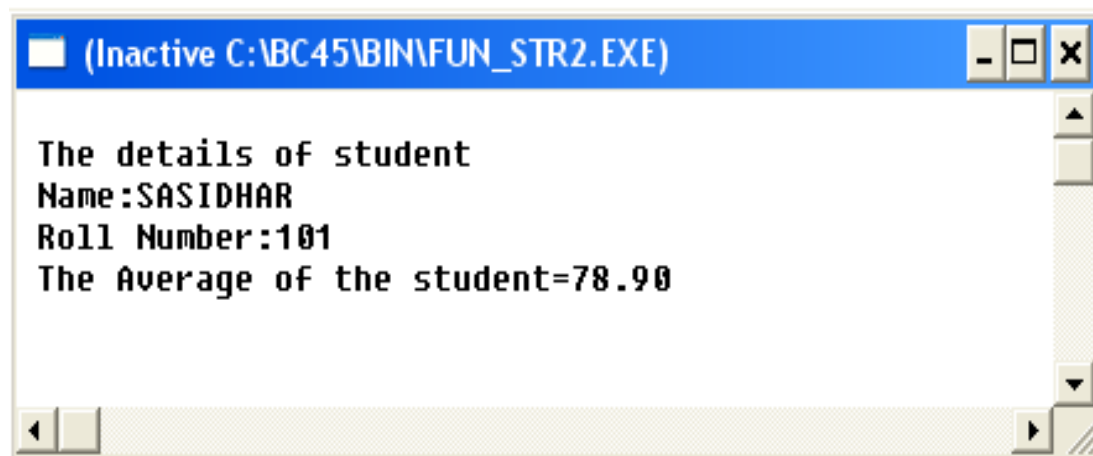
Returning Structure from a function

It is also possible that using return statement we can return structure to the calling function. Here the return type of the function and the return value both must be of type structure.

When the function call comes with an assignment to the structure variable, after executing the called function the structure is returned and it is copied into the structure variable in the assignment statement.

// Program illustrating function returning structure

```
#include<stdio.h>
struct student
{
    char name[10];
    int rno;
    float avg;
};
struct student fun( );
int main()
{
    struct student s1;
    s1=fun();
    printf("\n The details of student ");
    printf("\n Name:%s",s1.name);
    printf("\n Roll Number:%d",s1.rno);
    printf("\n The Average of the student=%.2f ",s1.avg);
    return 0;
}
struct student fun( )
{
    struct student s={"SASIDHAR",101,78.9};
    return s;
}
```

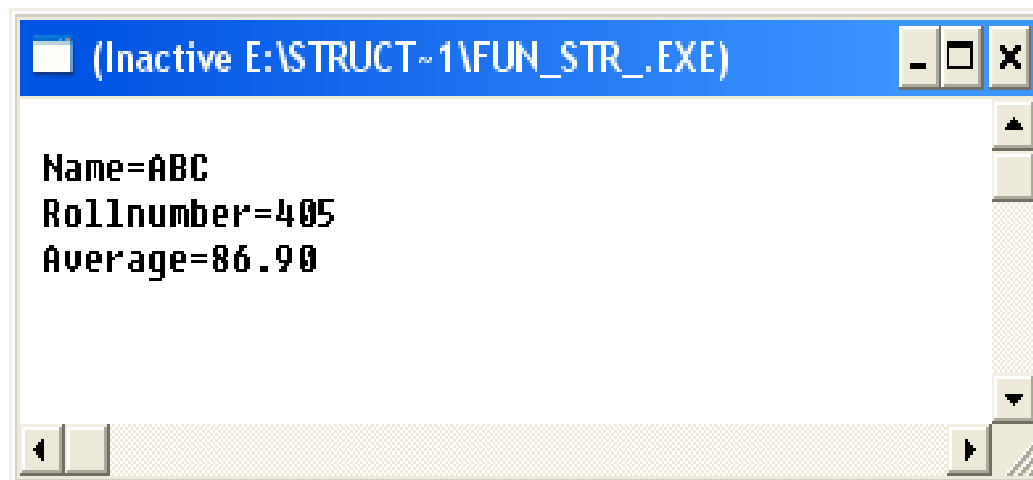


Passing Structures Through Pointers

We have a pointer pointing to a structure. such pointers are known as **"structure pointers"**. The third approach employs the concept of pointers to pass the structure as an argument. In this case, the address location of the structure is passed to the called function. The function can access indirectly the entire structure and work on it.

// C program illustrate Passing address of a structure variable

```
#include<stdio.h>
struct student
{
    char name[10];
    int rno;
    float avg;
};
void display(struct student * );
int main()
{
    struct student s1={"ABC",405,86.90};
    display(&s1);
    return 0;
}
void display(struct student *ptr)
{
    printf("\n Name=%s",ptr->name);
    printf("\n Rollnumber=%d \n Average=%.2f",ptr->rno,ptr->avg);
}
```



5.7 SELF-REFERENTIAL STRUCTURE

- ❖ A structure definition which includes at least one member as a pointer to the same structure is known as self-referential structure.
- ❖ Can be linked together to form useful data structures such as lists, queues, stacks and trees.
- ❖ Terminated with a NULL pointer (0).

The syntax for using the self referential structure as follows,

```
Struct tag_Name
{
    type1 member1;
    type2 member2;
    ....
    Struct tag_Name *next;
};
```

Example,

```
struct node
{
    int data;
    struct node *next;
} n1, n2;
```


Diagram of two self-referential structure objects linked together is shown in below Figure 5.6.

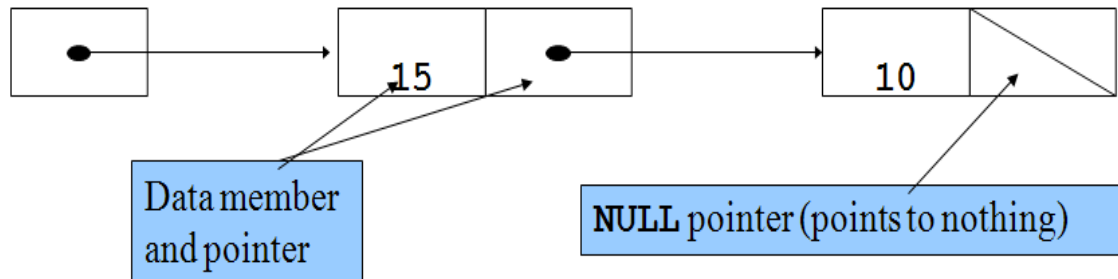


Figure 5.6 Self Referential Structure Object Linking

Here next points to an object of type node referred to as a link ties one node to another node.

5.8 UNIONS

- ❖ A union is one of the derived data type.
- ❖ Union is a collection of variables referred under a single name.
- ❖ The syntax, declaration and use of union is similar to the structure but its functionality is different.
- ❖ The major distinction between structure and union is, in terms of storage.
- ❖ In structure each member has its own storage location.
- ❖ Whereas all the members of a union use the same location.
- ❖ Although a union may contain many members of different types, it can handle only one member at a time.

The general format or syntax of a union definition is as follows,

```

union tag_name
{
    type1 member1;
    type2 member2;
    .....
    .....
};
    
```

Observe the following points while defining a union.

- ❖ union is the keyword which tells the compiler that a union is being defined.
- ❖ member1, member2, ... are called members(or fields) of the union.
- ❖ The members are declared within curly braces.
- ❖ The compiler allocates a piece of storage that is large enough to hold the largest variable type in the union.
- ❖ There should be semicolon at the end of closing braces.

A union variable can be declared same way as structure variable.

union tag_name var1, var2...;

A union definition and variable declaration can be done by using any one on the following Figure 5.7.

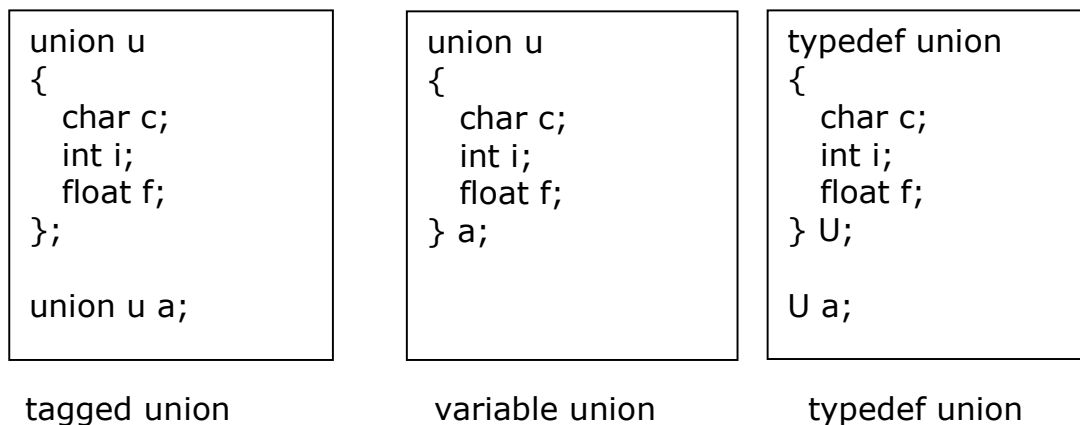


Figure 5.7 Types of Union Definitions

To access the members of union, the same syntax used to access various members of a structure can be used, by using the dot operator ("."). For above example, we can access various members of the union as shown below,

```
a.c
a.i
a.f
```

For the above example, union contains three members, each with a different data type. However, we can use only one of them at a time.

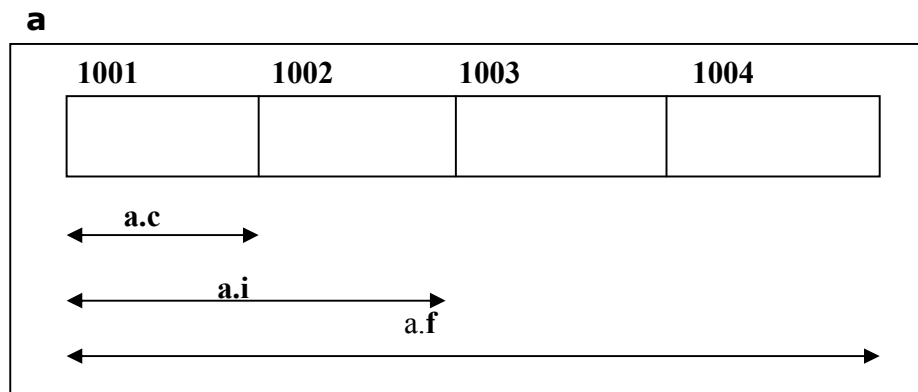
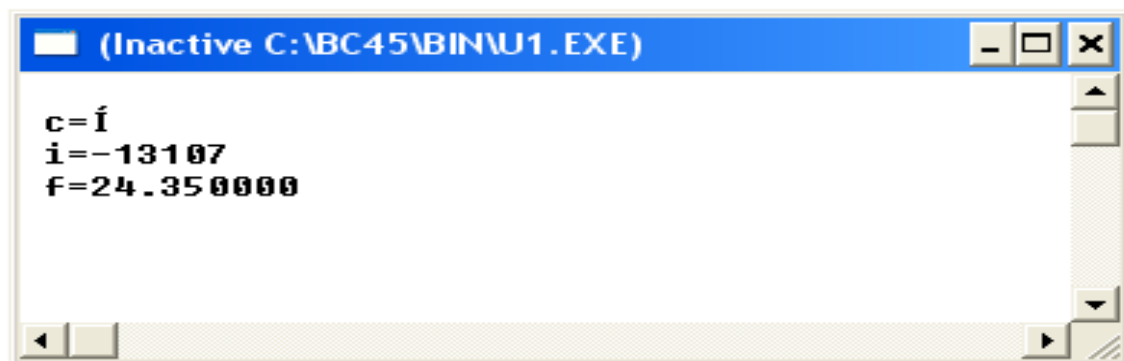


Figure 5.8: Memory Organization Union

In the above declaration, the member `f` requires 4 bytes which is the largest among the members. Figure 5.8 shows how all the three variables share the same address. The size of the structure here is 4 bytes.

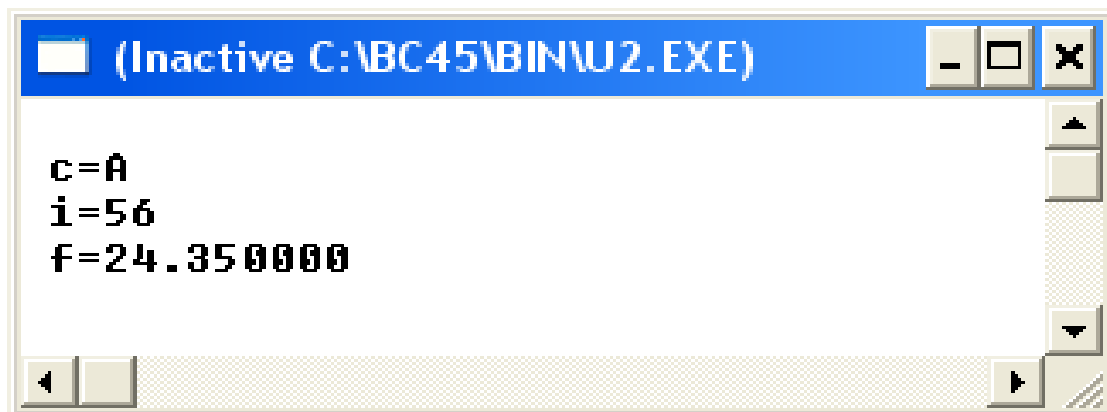
During accessing, we should make sure that we are accessing the member whose value is currently stored. For example look at the below program, would produces erroneous output (which is machine dependent).

```
#include<stdio.h>
union u
{
    char c;
    int i;
    float f;
};
int main()
{
    union u a;
    a.c='A';
    a.i=56;
    a.f=24.35;
    printf("\n c=%c",a.c);
    printf("\n i=%d",a.i);
    printf("\n f=%f",a.f);
    return 0;
}
```



In the above program we are initializing all the members of the union at a time, the member which is large enough to hold the largest variable type in the union occupies the memory, here float value is stored in all memory locations of the union.

```
#include<stdio.h>
union u
{
    char c;
    int i;
    float f;
};
int main()
{
    union u a;
    a.c='A';
    printf("\n c=%c",a.c);
    a.i=56;
    printf("\n i=%d",a.i);
    a.f=24.35;
    printf("\n f=%f",a.f);
    return 0;
}
```



A union creates a storage location that can be used by any one of its members at a time. When a different member is assigned a new value, the new value supersedes the previous members' value.

The above program illustrates the effective use of union memory locations.

We can also create a pointer to a union; the syntax is as follows,

```
union tag_name *ptr;
```

```
union u
{
    char c;
    int i;
    float f;
};

union u *ptr;
```

For the pointer variables, the indirection operator * and dot operator or selection operator -> can be used to access the members. For the above example, using pointer variable ptr, the various members of the union as shown below,

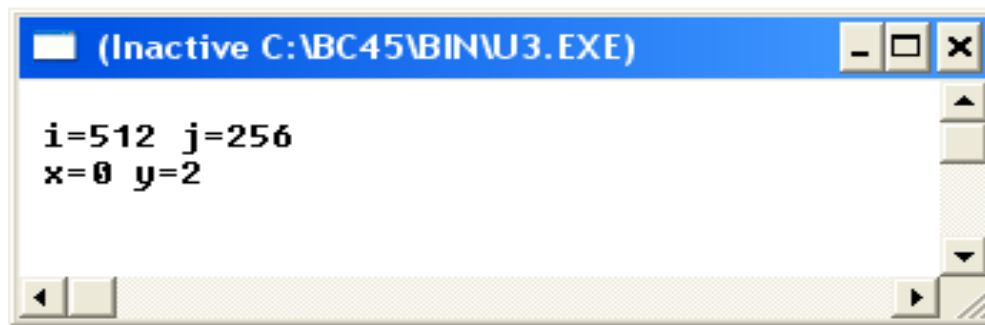
```
ptr -> c or (*ptr).c
ptr.i -> i or (*ptr).i
ptr.f -> f or (*ptr).f
```

structure with in union

It also possible union may also contain structures as members of it.
The following code illustrates this concept.

```
#include<stdio.h>
struct a
{
    int i;
    int j;
};
struct b
{
    char x;
    char y;
};
union u
{
    struct a aa;
    struct b bb;
};

int main()
{
    union u uu;
    uu.aa.i=512;
    uu.aa.j=256;
    printf("\n i=%d j=%d",uu.aa.i,uu.aa.j);
    printf("\n x=%d y=%d",uu.bb.x,uu.bb.y);
    return 0;
}
```



In the above program we have two structure declarations a and b followed by one union declaration u. Here the members of the union are two structure variables. Now the size of the union is 4 bytes, because the first structure contains two integer members so its size is 4 bytes, and the second structure contains two character members so its size is 2 bytes, therefore the size of the union is 4 bytes and entire memory locations are occupied by the first structure variable.

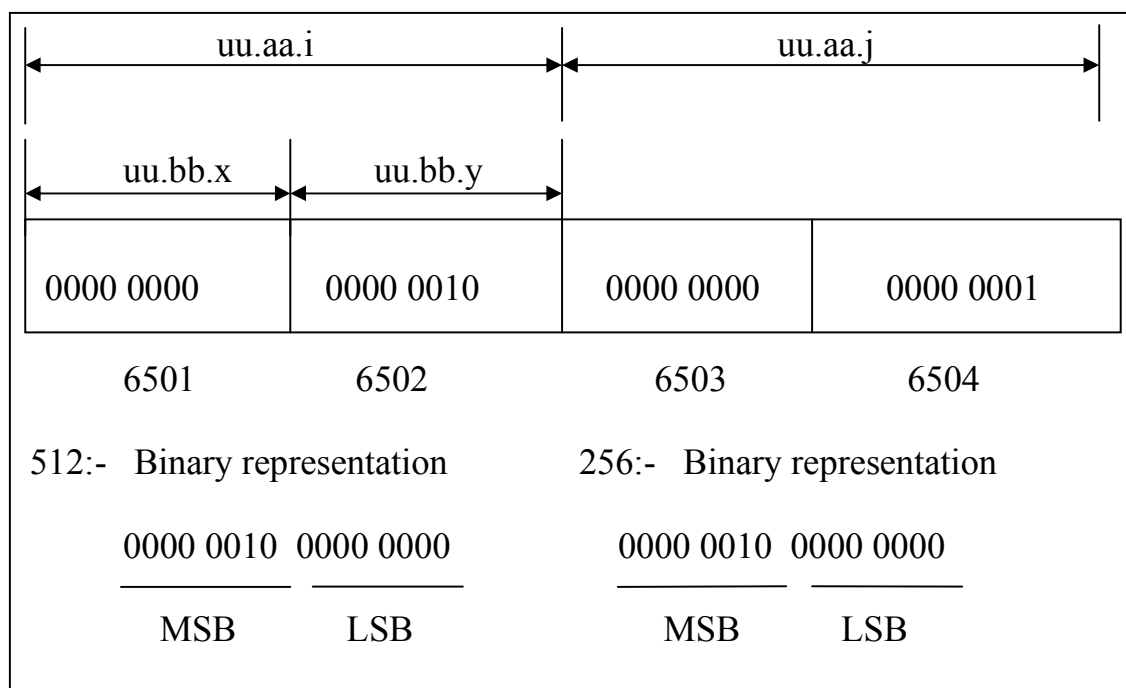


Figure 5.9: Sharing of Union Members

Difference between Array, Structure and Union

	Arrays	Structures	Unions
Keyword	...	struct	union
Defini- tion	An array is a homogeneous collection of data.	A structure is a collection of logically related elements, possibly of different types, having a single name.	A structure is a collection of logically related elements, possibly of different types, having a single name, shares single memory location.
Declarati on	data_type array_Name[s ize];	struct tag_name { type1 member1; type1 member2; };	union tag_name { type1 member1; type1 member2; };
Initializat ion	Done by separating list of values with comma (,), specified in Curly braces { }.	struct tag_name var; Same.	union tag_name var; Same.
Accessin g	Accessed by specifying array name with subscript	Accessed by specifying structure variable name.mem bername	Accessed by specifying Union variable name.mem bername
Memory Allocatio n	Each array element occupies memory, stored in contiguous locations.	Each member of the structure occupies unique location, stored in contiguous locations.	Memory is allocated by considering the size of largest member. All the members share the common location

Size	Size of the array is depending on the array type and size.	Size of the structure depends on the type of members, adding size of all members.	Size is given by the size of largest member storage.
Using pointers	sizeof (arr); An array elements values can be accessed by using de-referencing operator(*)	sizeof (st_var); Structure members can be accessed by using de-referencing operator dot operator and selection operator(->)	sizeof(un_variable); Same as structure.
	We can have array of structures or unions. here the array type is structure or union.	We can have arrays as a member of structures.	We can have array as a member of union.
	All elements can be accessed at a time	All members can be accessed at a time	Only one member can be accessed at a time.
	...	Nesting of structures is possible. It is possible union with in structure as a member.	Same. It is possible union may contain structure as a member

Table 5.3: Difference Between Array, Structure and Union

5.9 **TYPDEF**

C supports a feature known as "type definition" that allows users to define an identifier that would represent an existing data type. Its purpose is to redefine the name of an existing variable type.

The general syntax of the typedef is as follows,

```
typedef data_type IDENTIFIER;
```

where,

- ❖ `typedef` is the keyword tells the compiler about the type definition.
- ❖ `data_type` refers to an existing data type.
- ❖ `IDENTIFIER` refers the "new" name given to the data type.
- ❖ Usually, uppercase letters are used to make it clear that we are dealing with a renamed data type

Note that using `typedef`, we are not creating new data types. Instead we are creating only new name for the existing data type. These new data type names are called user-defined data types.

Suppose we want to store marks scored in various subjects in variables `sub1`, `sub2` and `sub3`. These variables can be declared as follows,

```
int sub1, sub2, sub3;
```

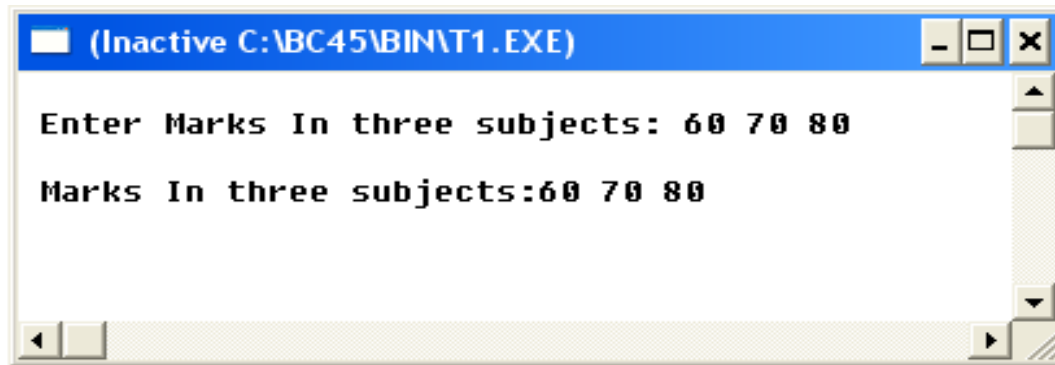
Using the user-defined data types, the variables can be declared as shown below,

```
typedef int MARKS;  
MARKS sub1, sub2, sub3;
```

Advantages

- ❖ Provides a meaningful way of declaring the variables.
- ❖ Increase the readability of the program.
- ❖ A complex declaration can be reduced to short and meaningful declaration.
- ❖ `typedef` is widely used while dealing with structures.

```
#include<stdio.h>
int main()
{
    typedef int MARKS;
    MARKS sub1, sub2, sub3;
    printf("\n Enter Marks In three subjects: ");
    scanf("%d%d%d", &sub1, &sub2, &sub3);
    printf("\n Marks In three subjects: ");
    printf("%d %d %d ", sub1, sub2, sub3);
    return 0;
}
```



typedef with structures

using typedef, it is efficient way to define a structure. It can be defined as follows,

```
typedef struct
{
    type1 member1;
    type2 member2;
    ...
} TYPE-ID;
```

Following are examples of typedef structure

```
struct employee
{
    char name [30];
    int age;
    float b_sal;
};
typedef Struct employee EMP;
EMP e1, e2;
```

```
typedef struct employee
{
    char name [30];
    int age;
    float b_sal;
} EMP;
EMP e1, e2;
```

members can be accessed same way as normal structure. But here we are creating new type. For the above example the type name is EMP.

typedef can also be used to rename pointer data types as shown below:

```
typedef struct employee
{
    char name [30];
    int age;
    float bs;
}
typedef struct employee *PEMP;
PEMP p;
```

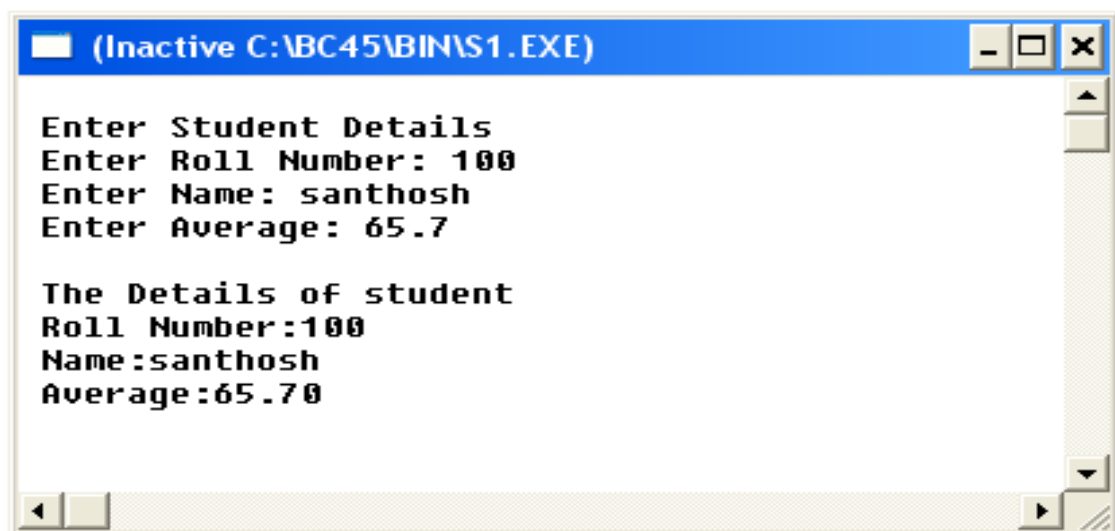
The below program illustrates the definition of structure with typedef, declaring variables and accessing members.

```
#include<stdio.h>
typedef struct
{
    int rno;
    char name[10];
    float avg;
} STUDENT;
int main()
{
    STUDENT s1;
    printf("\n Enter Student Details ");
    printf("\n Enter Roll Number: ");
    scanf("%d",&s1.rno);
    printf(" Enter Name: ");
```

```

    fflush(stdin);
    gets(s1.name);
    printf(" Enter Average: ");
    scanf("%f",&s1.avg);
    printf("\n The Details of student ");
    printf("\n Roll Number:%d",s1.rno);
    printf("\n Name:%s",s1.name);
    printf("\n Average:%.2f",s1.avg);
    return 0;
}

```



5.10 ENUMERATED TYPES

- ❖ We know that words speak more than numbers (as pictures speak more than words).
- ❖ For example we may use integers 1, 2, 3,.. 12 to represent months for easy programing.
- ❖ It is more readable and understandable if we replace these numbers by some meaningful and descriptive names such as Jan, Feb... Dec.
- ❖ This concept of replacing integers by some descriptive names gives rise to new data type called enumerated type.

- ❖ An enumerated data type is a user defined data type which can take the integer values from a list.
- ❖ These integer values are replaced by meaningful and descriptive names so as to enhance the readability of the program.
- ❖ These descriptive names are called **enumerators** or **enumerator constants**.

Declaring an Enumerated Type

To declare an enumerated type, we must declare its identifier and its values. Because it is derived from integer type, its operations are the same as for integers.

Syntax for defining an enumerated type is as follows,

```
enum type_Name
{
    member1;
    member2;
    ....
    ....
};
```

Where,

- ❖ **enum** is the keyword tells the compiler about enumerated type definition.
- ❖ **enum type_Name** together represent the user defined data type.
- ❖ **member1, member2...** are integer constants but represented using descriptive names. These are called enumerator constants or enumerators.
- ❖ The definition terminated with a semicolon.

The syntax for declaring the variables are shown below,

enum type_Name var;

Following are some of the examples of enumerator type:

```
enum color
{
    RED,
    BLUE,
    GREEN
};

enum color c1, c2;
```

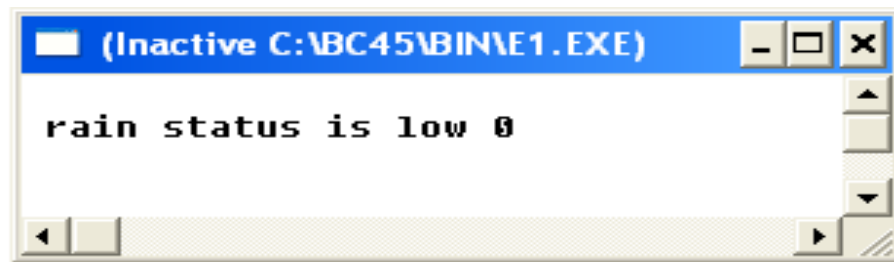
```
enum days
{
    SUNDAY,
    MONDAY,
    ...
    SATURDAY
} d1;
```

Assigning Values to Enumerated Types

After an enumerated variable has been declared, we can store values in it. While, the compiler automatically assigns values to enumerated types starting with 0, the next values are initialized with a value by adding 1 to previous value.

For example, to set up an enumerated type for the rain status, we could use the following program.

```
#include<stdio.h>
enum status
{
    low,medium,high
};
int main()
{
    enum status rain;
    rain=low;
    if(rain==low)
        printf("\n rain status is low %d",low);
    return 0;
}
```



We can override it and assign our own values, For example, for example to make JAN start with 1 we could use the following declaration

```
enum month
{
    JAN=1, FEB, MAR, APR, MAY, JUN, JUL, AUG, SEP, OCT, NOV, DEC
} m1;
```

Note that we don't have to assign initializes to every value. If we omit the initializes, the compiler assigns the next value by adding 1.

consider the following enumerated declaration,

```
enum days
{
    sun=3, mon, tue, wed=0, thu, fri, sat
} d1, d2;
```

Observe that first enumerator sun=3, so for the subsequent enumerators the compiler assigns 4, 5. Now the enumerators mon=4, tue=5.

But, observe that the enumerator, wed is initialized to 0. SO, the subsequent enumerators will have 1, 2, and so on. Now the enumerators thu=1, fri=2, sat=3.

Any manipulations done on integer variables can also done on the variables d1 and d2.

Look at the following statements,

```
w1= sun; // is valid assignment
w2= 5;   // is invalid assignment
```


It is also possible to associate typedef with enumerated type as shown below,

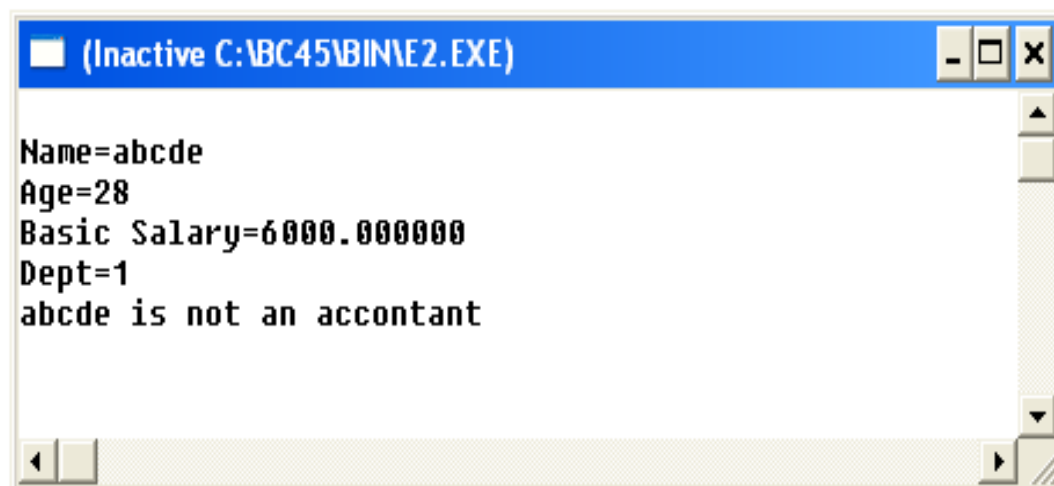
```
typedef enum
{
    sun=3, mon, tue, wed=0, thu, fri, sat
} DAYS;
```

DAYS d1, d2; // Declaring Variables

using typedef we can write more readable code. In this example, we can say the variables d1 and d2 are of type DAYS.

```
#include<stdio.h>
#include<string.h>
enum emp_dept
{
    assembly, manufacturing,
    accounts, stores
};
struct employee
{
    char name[30];
    int age;
    float bs;
    enum emp_dept department;
};
```

```
int main()  
{  
    struct employee e;  
    strcpy(e.name, "abcde");  
    e.age=28;  
    e.bs=6000.00;  
    e.department=manufacturing;  
    printf("\nName=%s", e.name);  
    printf("\nAge=%d", e.age);  
    printf("\nBasic Salary=%f", e.bs);  
    printf("\nDept=%d", e.department);  
    if(e.department== accounts)  
        printf("%s is an accountant", e.name);  
    else  
        printf("%s is not an accountant", e.name);  
    return 0;  
}
```



5.11 BIT FIELDS

If in a program a variable is to take only two values 1 and 0, we really need only a single bit to store it. If a variable is to take values from 0 to 3, then two bits are sufficient to store these values. So, we cannot waste entire integer to store such data.

There are several variables whose max values are small enough to pack into a single memory location. We can use 'bit fields' to store several values in a single integer.

A bit field is a set of adjacent bits whose size can be from 1 to 16 bits length. A word can therefore be divided into a number of bit fields. The name and size of bit fields are defined using structure. Since bit fields are defined within a structure, the various bits can be accessed in a way we access individual members of a structure.

The general form or syntax for bit field definition as follows,

```
struct tag_Name
{
    Unsigned int name1: bit_length;
    Unsigned int name2: bit_length;
    ...
    ...
    Unsigned int namen: bit_length;
};
```

Where,

- ❖ unsigned int is always used to define bit fields.
- ❖ bit_length is the number of bits used for the specified name.
- ❖ field name is followed by semicolon.

Remember that a signed bit field should have at least 2bits (1bit for sign). The bit_length is decided by the range of value stored. The largest value that can be stored is 2^{n-1} , where n is bit_length.

The internal representation of bit fields is machine dependent. Some machines store bits from left to right and others from right to left. The below Figure 5.10 illustrates the layout of bit fields, assuming a 16-bit word that is ordered from right to left.

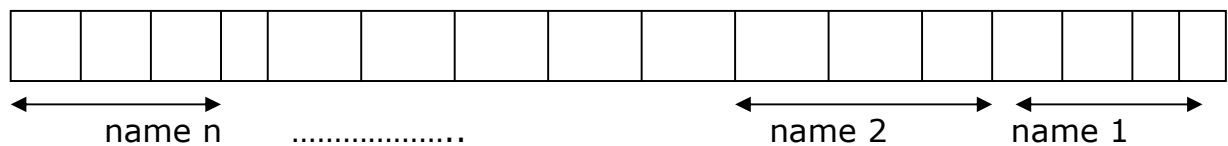


Figure 5.10: Bit Field Storage representation

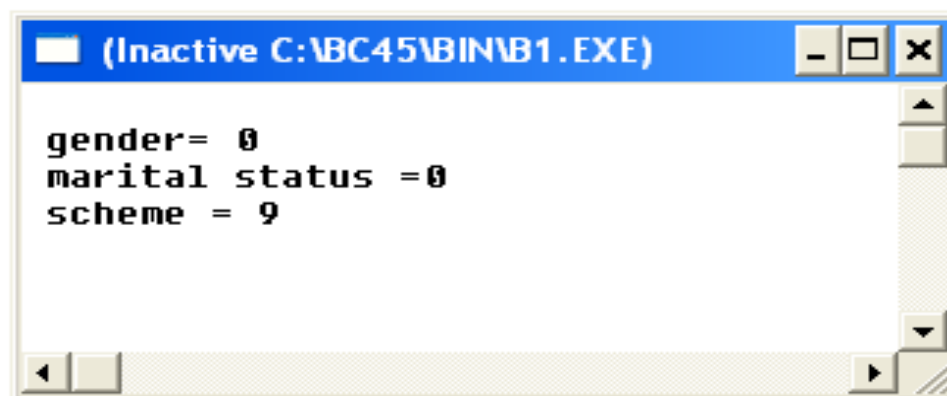
There are several specific points to observe:

- ❖ The first field always starts with first bit of the word.
- ❖ A bit can not overlap integer boundaries. This is, the sum of lengths of all fields in a structure should not be more than the size of a word. In case, it is more, the overlapping field is automatically forced to the beginning of the next word.
- ❖ There can be unnamed field declared with size. Example,
unsigned : bit_length;
 Such fields provide within a word.
- ❖ There can be unused bits in a word.
- ❖ We cannot take the address of a bit field variable.
- ❖ Bit fields cannot be arrayed.
- ❖ Bit fields should be assigned values that are the range of their size.

// C program illustrates Bit Fields

```
#include<stdio.h>
#define MALE 0;
#define FEMALE 1;
#define SINGLE 2;
#define MARRIED 3;
#define DIVORCED 4;
#define WIDOWED 5;
struct employee
{
    unsigned int gender: 1;
    unsigned int mar_stat: 2;
    unsigned int hobby: 3;
    unsigned int scheme: 4;
};
```

```
int main()
{
    struct employee e;
    e.gender = MALE;
    e.mar_stat = DIVORCED;
    e.hobby = 5;
    e.scheme = 9;
    printf("\n gender= %d", e.gender);
    printf("\n marital status=");
    printf("%d", e.mar_stat);
    printf("\n scheme = %d", e.scheme);
    return 0;
}
```



Exercise Programs

1. Write a program using structures to display the following information for each customer name, account number, street, city, old balance, current payment, new balance, account status.
2. Write a C program to accept records of the different states using array of structures. The structure should contain char state, population, literary rate, and income. Display the state whose literary rate is highest and whose income is highest.
3. Define a structure type struct **ABS**, which contains name, age, designation, and salary. Using this structure, write a C program to read this information for one person from the keyboard and print the same on the screen.
4. Write a C program using structure to read and print the student's records of a class with the following members.

Field Name	Data Type
name	string
reg no	integer
major	string
result	string

5. Write a C program to store the information of vehicles. Use bit fields to store the status information. Assume the vehicle object consists of type, fuel and model member fields. Assume appropriate number of bits for each field.
6. Write a C program to compute the monthly pay of 100 employees using each employee's name, basic-pay. The DA is computed as 52% of the basic pay. Gross-salary (Basic pay+DA). Print the employees name and gross salary.
7. Write a C program using structure to create a library catalogue with the following fields; access number, author's name, title of the book, year of publication, publisher's name, and price.
8. Define a structure that can describe a hotel. It should have members that include the name, address, grade, average room

- charge, and number of rooms. Write a C program to perform following operations:
- (a) To print out hotels of a given grade in order of charges.
 - (b) To print out hotels with room charges less than a given value.
9. Write a C program to read the information from the keyboard in which the "Employee" structure consists of employee name, code, designation and salary. Construct an array of structures that stores n employee's information and write a program to carry out operations like inserting a new entry, deleting entry.
10. Design a structure to store length in kilometers, meters and centimeters. Write a function to find the addition of two measurements by passing two measurements as arguments to a function.
11. A company markets Hardware items. Create a structure **"hwItem"** that stores the title of the item, its price, an array of three floats so that it can record the sale in rupees of a particular item for the last three months, category of the item and its original equipment manufacturer. Write a short program that provides facility to read N no. of items information, append new item, and displays all records.
12. The annual examination is conducted for 50 students for three subjects. Write a program to read the data and determine the following:
- (a) Total marks obtained by each student.
 - (b) The highest marks in each subject and the Roll No. of the student who secured it.
 - (c) The student who obtained the highest total marks.
13. Write a C program to accept records of the different states using array of structures. The structure should contain char state, population, literary rate, and income. Display the state whose literary rate is highest and whose income is highest.

ASSIGNMENT V

1. Define Structure and write the general format for declaring and accessing members.
2. a) How bits are manipulated using bit fields in 'C' explain?
b) What is the use of typedef in structure declaration?
3. a) How Structure elements are accessed using pointer? Which operator is used? Give an example
b) What is a self - referential structure? What kind of applications it is used? Give example.
4. a) Describe nested structures. Draw diagrams to explain nested structure.
b) How to compare structure variables? Give an example.
5. a) Distinguish between an array of structures and an array within a structure. Give an example each.
b) Write a program to use structure within union. Display the contents of structure elements.
6. Compare arrays, structures and unions.